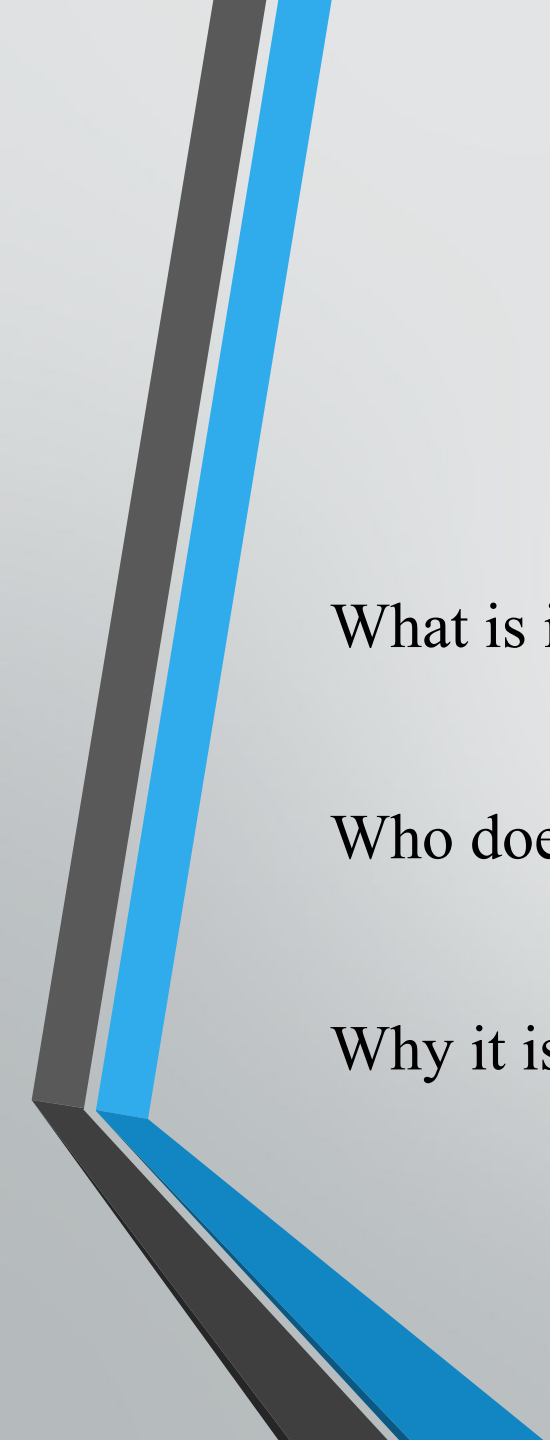




Introduction

What is it?

Who does it?

Why it is important?

What is Software

- Software is a set of instructions executed by a computer which are designed to perform a particular task.
- Generally these set of instructions is termed as program.
- Two Categories
 - System Software
 - Application Software

What is Software Testing?

- Software testing is **process** of analyzing or operating software for the purpose **of finding bugs**.
- Testing is the process of identifying defects, where a defect is any variance between **actual and expected results**.
- Soft. Testing is the process used to help identify the **correctness, completeness, security and quality** of developed computer software.

What is Software Testing

- Testing is **questioning** a product in order to evaluate it.

Fundamental Objective

- The Fundamental objectives of testing to find defects, **as early as possible** and get them fixed.

Testing Objectives

- 1. Testing is a process of executing a program with the intend of finding an error.
- 2. A good test case is one that has high probability of finding an undiscovered error.
- 3. A successful test is one that uncovers an as yet undiscovered error.

Testing Principles

- 1. The goal of testing is to find the defects before customer finds them out.
- 2. Always understand the reason behind the testing.
- 3. Testing is to be carried out throughout the software life cycle & not simply at the end of the software development process.
- 4. Testing should help for both defect detection & defect prevention.
- 5. Testing is a process that need to be carried out constantly.

Why Testing is important?

- 1. Testing is process that requires more efforts than any other software engineering activity.
- Testing is set of activities that can be planned.

Challenges in Testing

1. Requirements are not clear, complete & testable.
2. Lack of technical knowledge
3. Requirements are wrongly documented
4. Errors remains hidden

Participants in Testing – Who.?

- Customers
- Users
- Developers
- Testers
- Senior Management
- Auditors

A strategic approach to software Testing

- 1. To perform effective testing, you should conduct effective technical reviews.
- 2. Testing begins at component level and works outwards toward the integration
- 3. Different testing techniques can be applied at different point of time.
- 4. Testing & Debugging are different activities.

Verification & validation

Verification:

"Are we building the product right".

The software should conform to its specification.

Validation:

"Are we building the right product".

The software should do what the user really wants.

Verification

- Disciplined approach to evaluate whether a software product fulfils the requirements or condition imposed on them (**Are we doing the job right?**)
- It is also called static testing.
- Done by systematically reading the contents of a software product with the intention of detecting defects.
- It helps in identifying not only presence of defects but also their location.
- A ‘filter’ applied at various points during the SDLC to ‘purify’ the product as it progresses through various phases.

Verification

- Method: Walkthrough, Inspection and Review of each software product.
- This process helps in prevention of potential bugs, which may cause in failure of the project.

Walkthrough

- An informal process, initiated by the author of a software product to a colleague for assistance in locating defects and for suggesting improvements
- During this meeting, the presenter/ author introduces the material to all the participants in order to make them familiar with it.

Inspection

- A thorough, word- by-word checking of a software product (or part of a product) with the intention of :
 1. Locating defects
 2. Confirming traceability of relevant requirements
 3. Checking for conformance to relevant standards and conventions
- A typical inspection session takes around 90-120 minutes and typically they detect 30-70% of logic design and coding errors.

Review

- Also term as Follow-Up
- It is a subsequent examination of a product for the purpose of monitoring earlier changes.
- Formal examination.
- Review is defined as a process in which one or more persons checks changed documents or data to determine if the changes are correct.

Validation

- Disciplined approach to evaluate whether the final, as built software product fulfils its specific intended use.
- Also called as Dynamic Testing
- Done by systematically testing a software product with the intention of finding defects
- Helps in identifying the presence of defects, not their location
- Method: Testing each soft. Product at each phase of life cycle using test plan, test cases. (Unit, Integration, System & Acceptance Testing)

Verification Vs Validation

Verification	Validation
1. Verification is a static practice of verifying documents, design, code and program.	1. Validation is a dynamic mechanism of validating and testing the actual product.
2. It does not involve executing the code.	2. It always involves executing the code.
3. It is human based checking of documents and files.	3. It is computer based execution of program.
4. Verification uses methods like inspections, reviews, walkthroughs, and Desk-checking etc.	4. Validation uses methods like black box (functional) testing, gray box testing, and white box (structural) testing etc.
5. Verification is to check whether the software conforms to specifications.	5. Validation is to check whether software meets the customer expectations and requirements.
6. It can catch errors that validation cannot catch. It is low level exercise.	6. <i>It can catch errors that verification cannot catch. It is High Level Exercise.</i>
7. Target is requirements specification, application and software architecture, high level, complete design, and database design etc.	7. Target is actual product-a unit, a module, a bent of integrated modules, and effective final product.
8. Verification is done by QA team to ensure that the software is as per the specifications in the SRS document.	8. Validation is carried out with the involvement of testing team.
9. It generally comes first-done before validation.	9. It generally follows after verification .

Organizing for Software Testing

- Testing needs to be conducted from the beginning of SDLC.
- It is desired that software developer should test individual unit of software system for its proper functioning.
- Independent test group(ITG)

Criteria for Completion of Testing

- Entry Criteria: defined as “ specific condition or on going activities that must be present before a process can begin.”
- Exit Criteria: defined as “specific condition or on going activities that should fulfilled before completing the software testing life cycle”.

Strategic Issues

- Specify product requirements in a quantifiable manner long before testing commences.
- State testing objectives explicitly.
- Understand the users of the software and develop a profile for each user category.
- Develop a testing plan that emphasizes “rapid cycle testing”.
- Build “robust” software that is designed to test itself.
- Use effective technical reviews as a filter prior to testing.
- Conduct technical reviews to assess the test strategy and test case themselves.
- Develop a continuous improvement approach for testing process

• 1. which of the following term describe testing? **MCQ**

- A. Evaluating deliverables to find errors.
- B. A stage of all project
- C. Finding broken code
- D. None of these

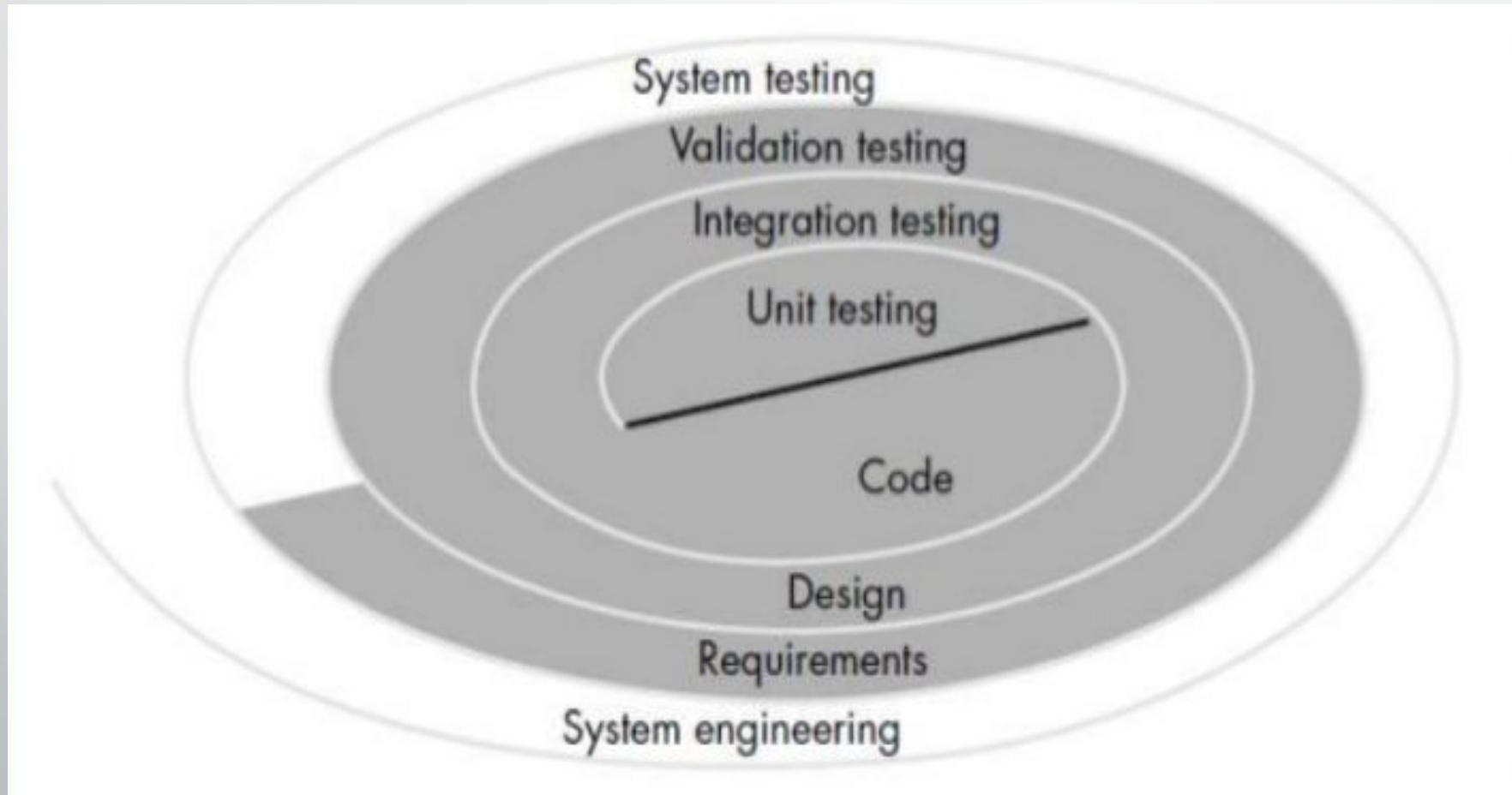
2. Testing of individual components by the developer are comes under_____.

- A. Integration testing
- B. Validation testing
- C. Unit testing

3. The testing have been stopped when _____.

- A. The faults have been fixed
- B. All the tests run
- C. The time completed
- D. The risk are resolved

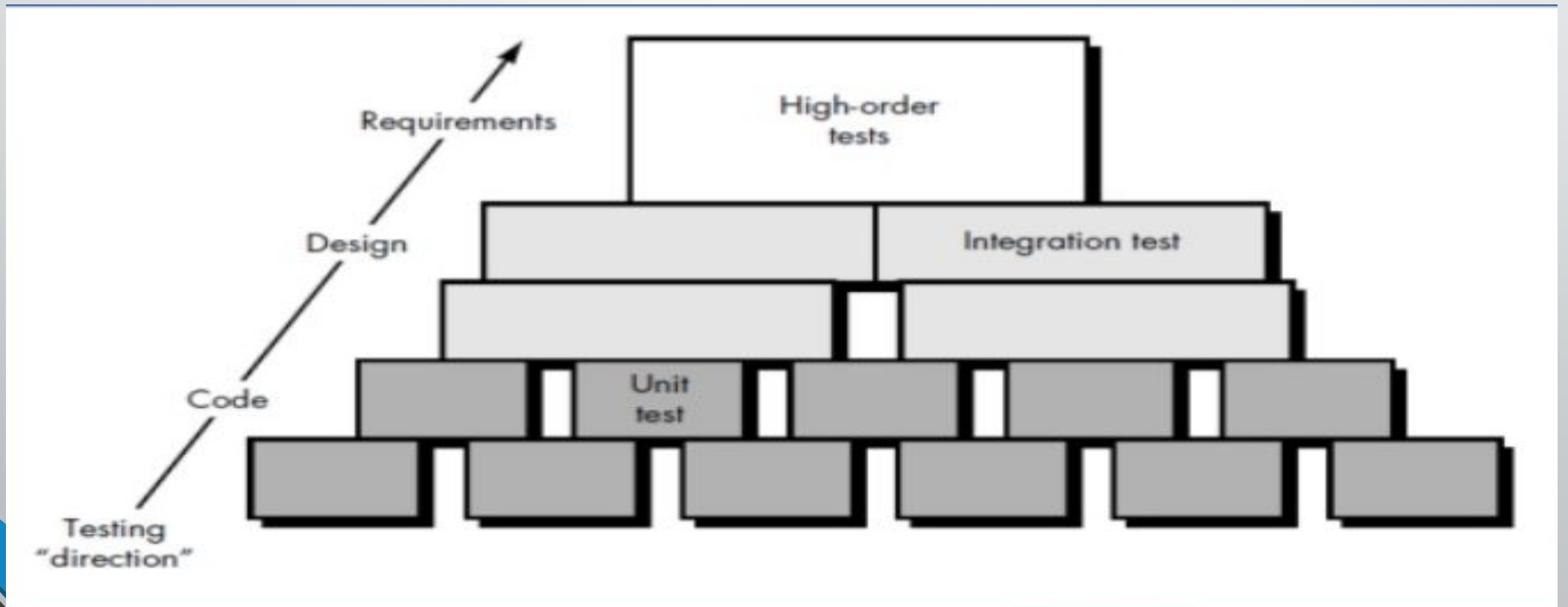
Software Testing Strategy-Big Picture



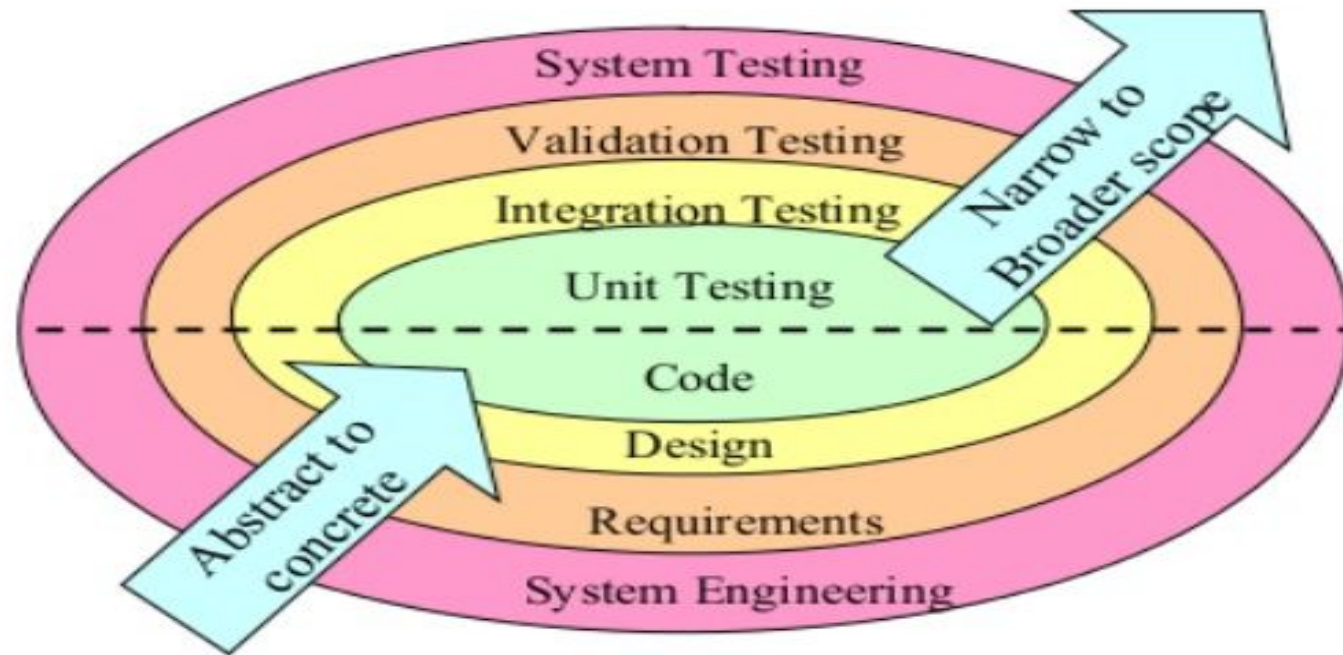
Software Testing Strategy-Big Picture

- Unit testing begins at the vortex of the spiral and concentrates on each unit (i.e., component) of the software as implemented in source code.
- Testing progresses by moving outward along the spiral to integration testing, where the focus is on design and the construction of the software architecture.
- Taking another turn outward on the spiral, we encounter validation testing, where requirements established as part of software requirements analysis are validated against the software that has been constructed.
- Finally, we arrive at system testing, where the software and other system elements are tested as a whole

Procedural view of Testing Strategy



Test Strategies for Conventional Software



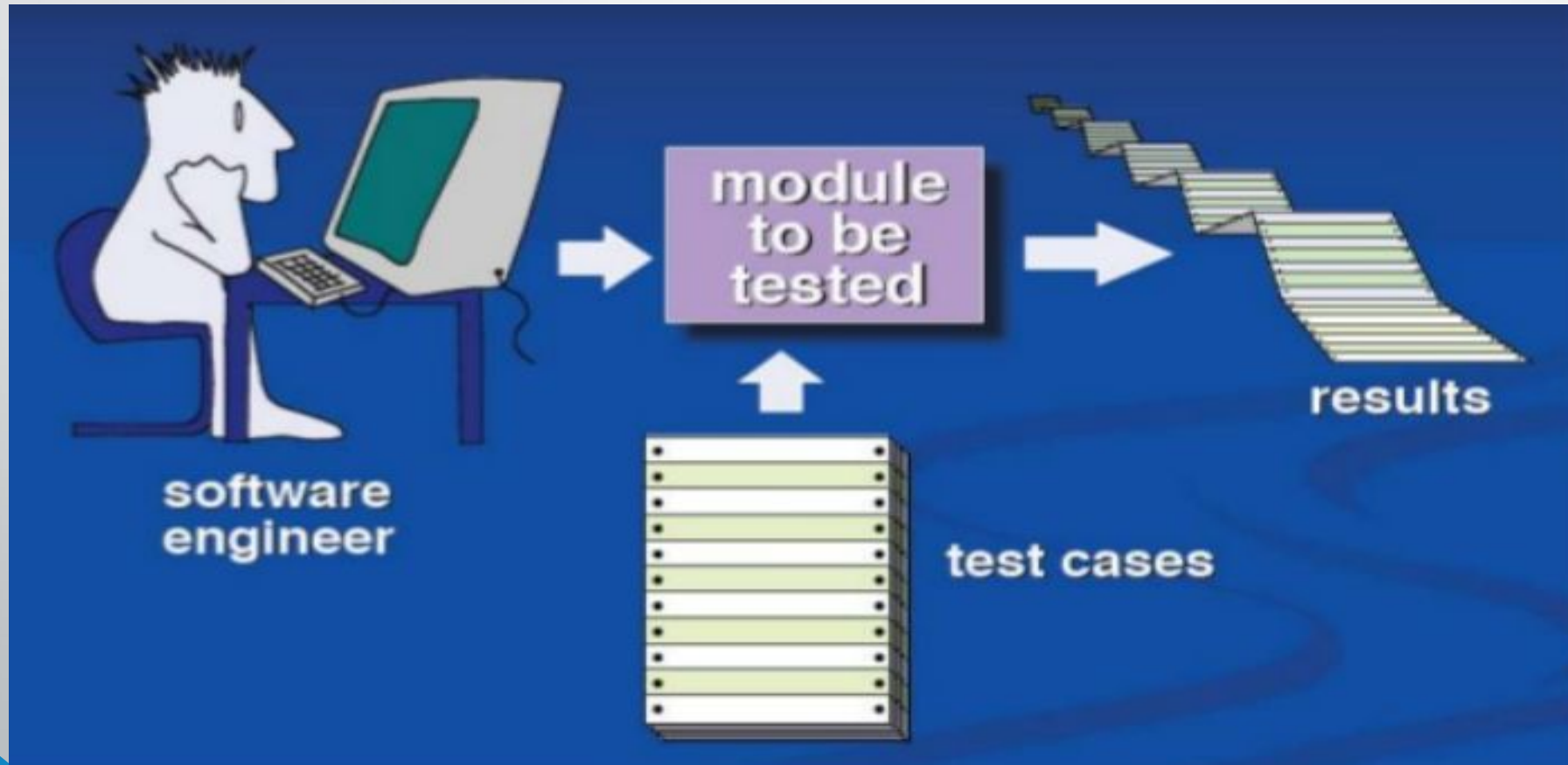
Levels of Testing for Conventional Software

- **Unit testing:** Concentrates on each component/function of the software as implemented in the source code
- **Integration testing:** Focuses on the design and construction of the software architecture
- **Validation testing:** Requirements are validated against the constructed software
- **System testing:** The software and other system elements are tested as a whole

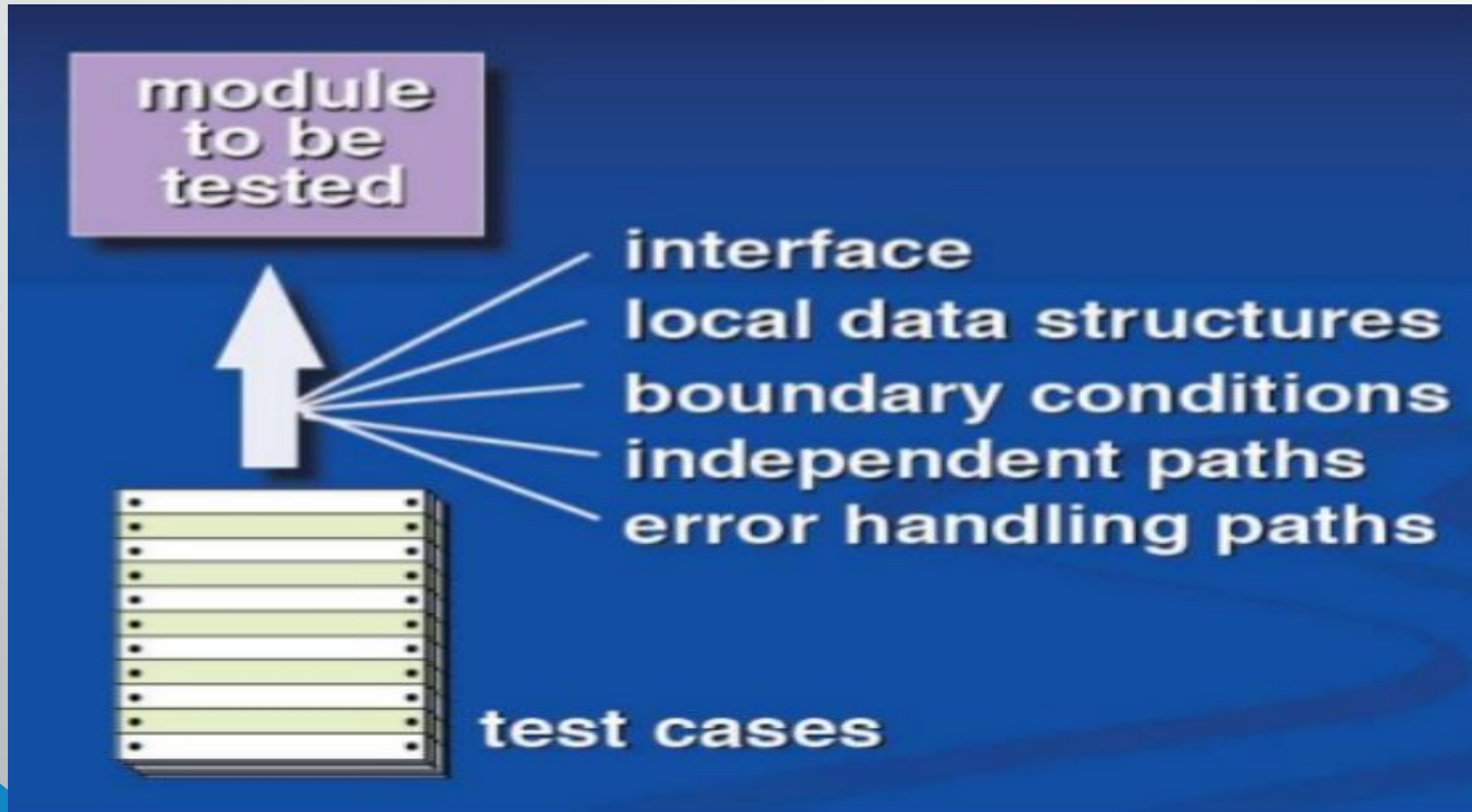
Testing Strategy applied to Conventional Software

- **Unit testing:** Exercises specific paths in a component's control structure to ensure complete coverage and maximum error detection • Components are then assembled and integrated
- **Integration testing:** Focuses on inputs and outputs, and how well the components fit together and work together
- **Validation testing:** Provides final assurance that the software meets all functional, behavioral, and performance requirements
- **System testing:** Verifies that all system elements (software, hardware, people, databases) mesh properly and that overall system function and performance is achieved

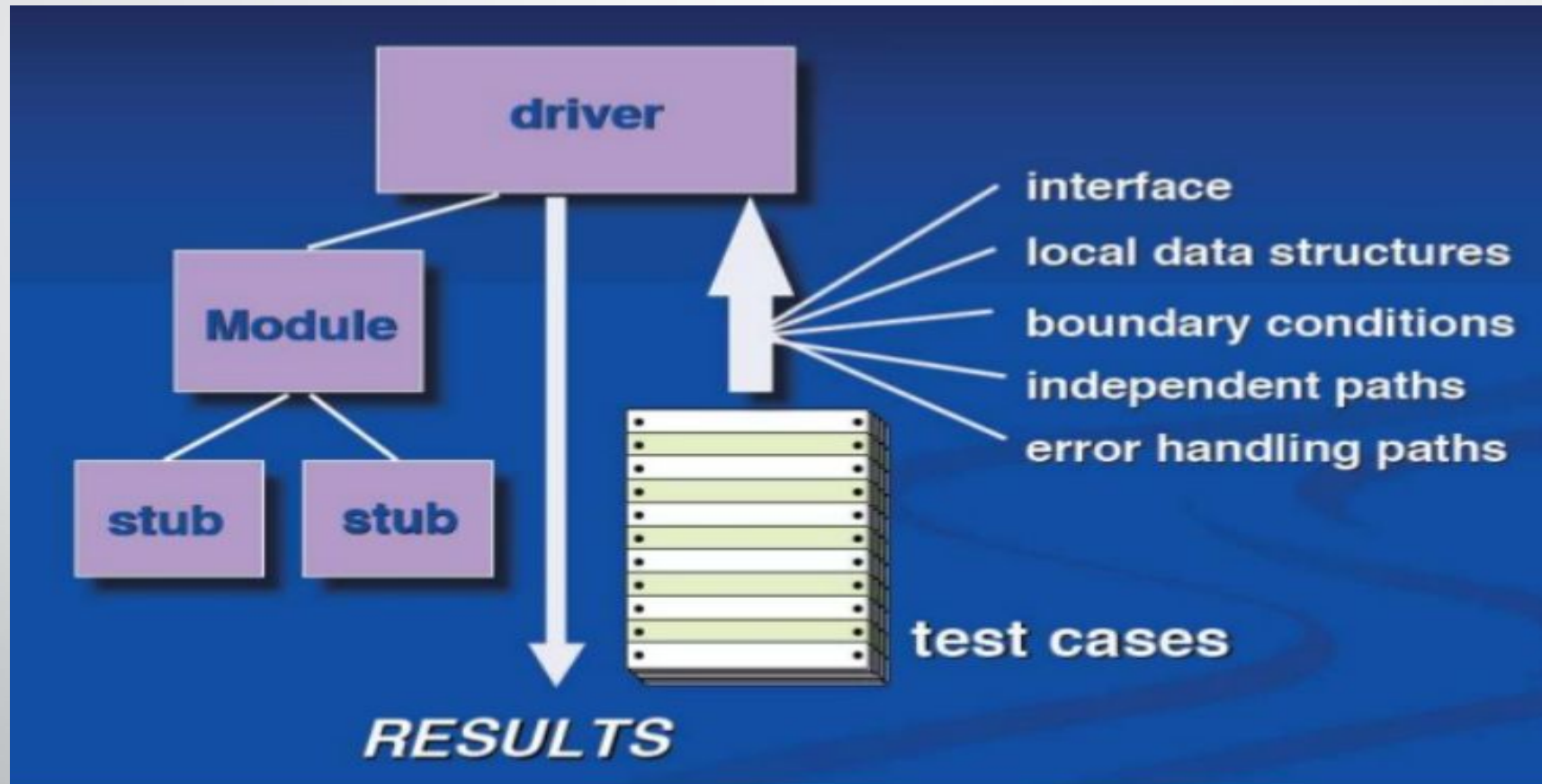
Unit testing



Unit testing



Unit testing



Integration Testing

- A group of dependent components are tested together to ensure quality of their integration unit.
- **Objective :** is to take unit tested components & build a program structure that has been dictated by software design.

Integration Testing

- Focus is to uncover errors in:
 - Design & construction of software architecture
 - Integrated functions or operations at subsystem level
 - Interfaces & interactions between them
 - Resource integration & environment integration

Integration Testing Strategies

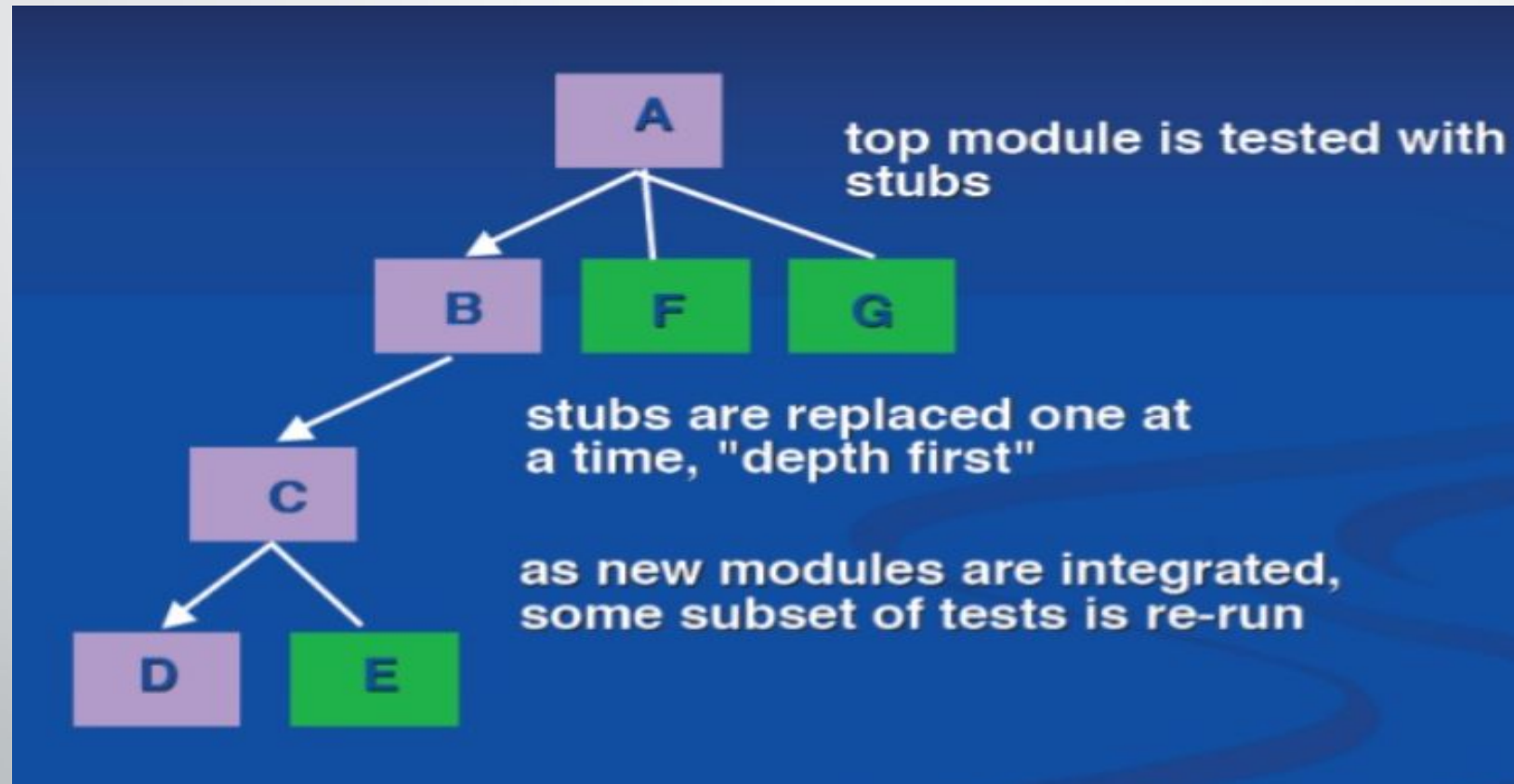
1. Incremental Integration :

- Top-down Integration
- Bottom-Up Integration

2. Non-Incremental Integration

- Big Bang

Top Down Integration Testing



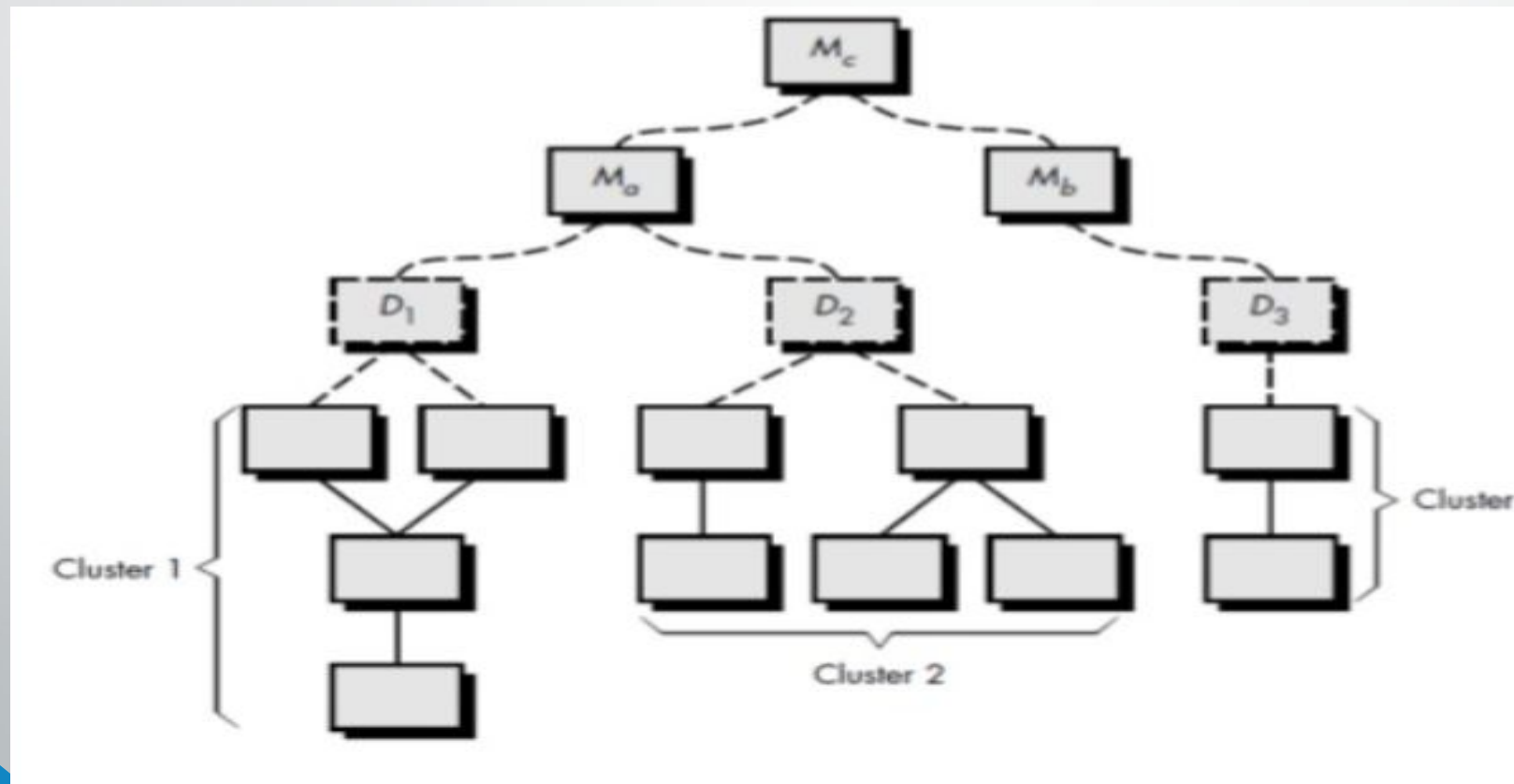
Example

Approve leave

Create a leave
Report

Stub

Bottom Up integration Testing



Example

Apply leave

Approve leave

Driver
(calling module)

Regression Testing

- It is defined as a type of software testing to confirm that a recent program or code change has not adversely affected existing features.
- It is nothing but a full or partial selection of already executed test cases which are re-executed to ensure existing functionalities work fine.

Regression Testing

- 1. representative sample of existing test cases is used to exercise all software functions.
- 2. Additional test cases focusing software functions likely to be affected by the changes.
- 3. Test cases that focus on the changed software components.

Smoke Testing

- It is an integration testing approach that is commonly used when product software is developed.
- It is designed as a mechanism for time-critical projects, allowing the software team to assess the project on a frequent basis.

Smoke Testing

- 1. Software components already translated into code are integrated into a “build” can be data files, libraries , reusable modules or program components.
- 2.A series of test are designed to expose errors from build so that “build” performs its functioning correctly.
- The “build” is integrated with the other builds and the entire product is smoke tested daily.

System Testing

- It is a series of tests conducted to fully the computer based system.

Types:

1. Recovery testing
2. Security testing
3. Stress testing
4. Performance testing

Performance Testing

- Performance Testing is a software testing process used for testing the speed, response time, stability, reliability, scalability and resource usage of a software application under particular workload.

Recovery Testing

- Recovery Testing is software testing technique which verifies software's ability to recover from failures like software/hardware crashes, network failures etc
- The purpose of Recovery Testing is to determine whether software operations can be continued after disaster or integrity loss
- Recovery testing involves reverting back software to the point where integrity was known and reprocessing transactions to the failure point.

Security testing

- SECURITY TESTING is a type of Software Testing that uncovers vulnerabilities, threats, risks in a software application and prevents malicious attacks from intruders.
- The purpose of Security Tests is to identify all possible loopholes and weaknesses of the software system which might result in a loss of information, revenue, reputation at the hands of the employees or outsiders of the Organization.

Stress Testing

- Stress Testing is a type of software testing that verifies stability & reliability of software application.
- The goal of Stress testing is measuring software on its robustness and error handling capabilities under extremely heavy load conditions and ensuring that software doesn't crash under crunch situations.
- It even tests beyond normal operating points and evaluates how software works under extreme conditions.

MCQ

1. What are the different levels of Testing?

- A. Integration testing
- B. Unit testing
- C. System testing
- D. All of the above

2. What is the best time to perform Regression testing?

- A. After the software has been modified
- B. As frequently as possible
- C. When the environment has been modified
- D. Both option a & c

3. Which is not the right approach of Incremental testing approach?

- A. Big bang approach
- B. Top-down approach
- C. Functional incrimination
- D. Bottom-up approach

Testing Strategies for object Oriented Software

- With object-oriented software, you can no longer test a single operation in isolation (conventional thinking).
- Traditional top-down or bottom-up integration testing has little meaning
- Class testing for object-oriented software is the equivalent of unit testing for conventional software.
 - Focuses on operations encapsulated by the class and the state behavior of the class
- Drivers can be used
 - To test operations at the lowest level and for testing whole groups of classes
 - To replace the user interface so that tests of system functionality can be conducted prior to implementation of the actual interface
- Stubs can be used
 - In situations in which collaboration between classes is required but one or more of the collaborating classes has not yet been fully implemented

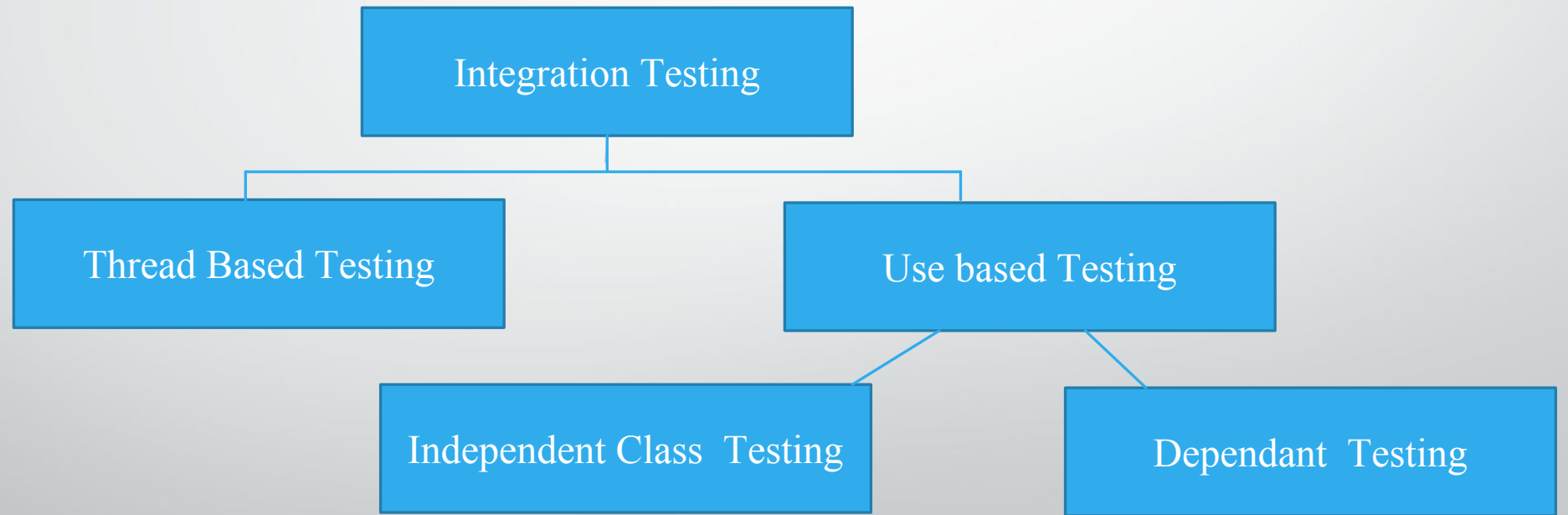
Unit testing in OO Context

- It involves testing for each class and each instance of a class (object) packages, attributes (i.e. data) and the operations (also known as methods or services) that manipulate these data.
- Rather than testing an individual module, the smallest testable unit is the encapsulated class or object. This is so because a class can contain a number of different operations and a particular operation may exist as part of a number of different classes.
- Consider a class hierarchy in which an operation X is defined for the superclass and is inherited by a number of subclasses.
- Each subclass uses operation X, but it is applied within the context of the private attributes and operations that have been defined for the subclass.

Unit testing in OO Context

- Because the context in which operation X is used varies in subtle ways, it is necessary to test operation X in the context of each of the subclasses.
- Class testing for OO software is the equivalent of unit testing for conventional ware.
- Unlike unit testing class testing for OO software is driven by the operations encapsulated by the class and the state behavior of the class.

Integration Testing in OO Context



Integration Testing in OO Context

- There are two different strategies for integration testing of OO system
- The first i.e. thread-based testing, integrates the set of classes required to respond to one input or event for the system. Each thread is integrated and tested individually. Regression testing is applied to ensure that no side effect occur.
- The second integration approach i.e. the use-based testing, begins the construction of the system by testing those classes (called independent classes) that use very few (if any) of server classes. After the Independent classes are tested, the next layer of classes, called dependent classes, that use the independent classes are tested. This sequence of testing layers of dependent classes continue until the entire system is constructed.
- Cluster testing is one step in the integration testing. Here a cluster of collaborating classes (determined by examine the CRC and object-relationship model) is exercised by designing test cases that attempt to uncover errors in the collaborations.

Test Strategies for Web Apps

1. The content model must be reviewed in order to uncover the errors.
2. The interface model is reviewed to ensure that all use cases can be accommodated.
3. The user interface is tested to uncover errors in presentation and / or navigation mechanics.
4. The design model for the webapp is reviewed to uncover navigation errors.
5. Each functional component is unit tested.

Test Strategies for Web Apps

6. Navigation throughout the architecture is tested.
7. The webapp is implemented in a variety of different environmental configurations and is tested for compatibility with each configuration.
8. Security tests are conducted in an attempt to exploit vulnerabilities in the WebApp or within its environment.
9. Performance tests are conducted.
10. The WebApp is tested by a controlled and monitored population of end users.

Validation Testing in an OO Context

- The validation of OO software focuses on user-visible actions and user-recognisable output from the system. To assist in the derivation of validation testing, the tester should
- work upon the use-cases that are part of the analysis model. The use-case provides a scenario that has a high likelihood of uncovered errors in the user interaction requirements.
- Conventional black-box testing methods can be used to drive validation tests.
- In addition, test cases may be derived from the object-behavior model and from event flow diagram created as part of OOA.

Validation Testing

- The process of evaluating software during the development process or at the end of the development process to determine whether it satisfies specified business requirements.
- Validation Testing ensures that the product actually meets the client's needs. It can also be defined as to demonstrate that the product fulfills its intended use when deployed on appropriate environment.

Validation Testing

- Main focus is to uncover errors in:
 - System input/ output
 - System functions & information data
 - System interfaces with external parts
 - User Interfaces
 - System behaviour & performance



Why Validation Testing is Important?

- To ensure customer satisfaction
- To be confident about the product
- To fulfill the client's requirement until the optimum capacity
- Software acceptance from the end-user

Validation Testing

- Validation testing follows integration testing
- The distinction between conventional and object-oriented software disappears
- Focuses on user-visible actions and user-recognizable output from the system
- Demonstrates conformity with requirements
- Designed to ensure that:
 - All functional requirements are satisfied
 - All behavioral characteristics are achieved
 - All performance requirements are attained
 - Documentation is correct
 - Usability and other requirements are met (e.g., transportability, compatibility, error recovery, maintainability)
- After each validation test :
 - The function or performance characteristic conforms to specification and is accepted
 - A deviation from specification is uncovered and a deficiency list is created

A configuration review or audit ensures that all elements of the software configuration have been properly developed, cataloged, and have the necessary detail for entering the support phase of the software life cycle

Alpha and Beta Testing

- Alpha testing Conducted:
 - at the developers site by end users
 - Software is used in a natural setting with developers watching intently
 - Testing is conducted in a controlled environment
- Beta testing
 - Conducted at end-user sites Developer is generally not present
 - It serves as a live application of the software in an environment that cannot be controlled by the developer
 - The end-user records all problems that are encountered and reports these to the developers at regular intervals

After beta testing is complete, software engineers make software modifications and prepare for release of the software product to the entire customer base

Alpha Vs Beta Testing

Alpha Testing	Beta Testing
Performed at developer's site	Performed at end user's site
Performed in controlled environment as developer is present	Performed in uncontrolled environment as developer is not present
Less probability of finding of errors as it is driven by developers	High probability of finding of errors as end user can use it the way he wants
It is not considered as live application	It is considered as live application
It is done during implementation phase of software	It is done as pre-released of software
Less time consuming as developer can make necessary changes in given time	More time consuming as user has to report bugs id any via appropriate channel

Validation Test Criteria

1. The function or performance characteristics conforms to specification and is accepted or
2. A derivation from specification is represented

Configuration Review

- The main purpose of this review is to ensure that all the elements of the software configuration have been properly developed and catalogued.
- The configuration review is also called as **Audit**.

MCQ

1. Which one is the main focus of acceptance testing?

- A. Testing the system with other systems
- B. Testing for a Business perspective
- C. Finding faults in the system
- D. Testing the system with other systems

2. Testing of software with actual data & in actual environment is known as ____.

- A. Regression testing
- B. Beta testing
- C. Alpha testing
- D. None of these

3. Beta testing is done at ____.

- A. developer's end
- B. User's end
- C. User's & developer's end